

Top 60 Automation Testing Interview Questions & Answers

Selenium | API | CI/CD | Python | Playwright



1. What is your automation strategy in your current project?

I follow a structured approach: analyze test cases → identify stable scenarios → design framework → develop scripts → integrate with CI/CD → maintain scripts after every release.

2. Which automation framework have you built? Explain architecture.

A Hybrid POM Framework containing:

- Page Object Model
- TestNG
- Data-driven (Excel/JSON)
- Utilities (waits, logs, screenshots)
- Reporting (Allure/Extent)
- CI integration

3. How do you decide which test cases to automate?

Automate:

- High regression scenarios
- Repetitive flows
- Stable modules
- Data-heavy tests
- High-risk business flows

Avoid:

- UI-changing modules
- One-time tests
- Unstable requirements

4. Explain your Selenium automation workflow.

Driver setup → browser launch → page methods → validations (assertions) → reporting → teardown.

5. How do you handle dynamic web elements?

Using:

- Dynamic XPath (starts-with, contains)
- CSS patterns
- Explicit waits
- JavaScript executor

6. What are the different waits used in Selenium?

- Implicit wait
- Explicit wait
- Fluent wait

7. What is Synchronization?

Handling timing issues between script and web page loading using waits.

8. What is POM (Page Object Model)? Why use it?

A design pattern that stores locators + methods in separate classes.

Advantages: reusability, maintainability, readability.

9. Difference between POM vs PageFactory.

- **POM** → manual element initialization
- **PageFactory** → uses `@FindBy` + `initElements()`

10. What are TestNG annotations used in your framework?

`@BeforeMethod`, `@AfterMethod`, `@Test`, `@DataProvider`, `@Parameters`, `@Listeners`.

11. How do you run tests in parallel?

Using TestNG XML: `parallel="tests"` | `"classes"` | `"methods"` or Selenium Grid.

12. What is Selenium Grid?

A tool to run tests on different machines & browsers in parallel.

13. How do you perform cross-browser testing?

Using `WebDriverManager` or Grid to run scripts on Chrome, Edge, Firefox.

14. How do you capture screenshots for failed tests?

Using `ITestListener` + `TakesScreenshot` interface.

15. What is the difference between Assert and SoftAssert?

- **Assert** → stops execution
- **SoftAssert** → continues & logs failures

16. What challenges have you faced in automation?

- Flaky locators
- Timeout issues
- Frequent UI changes
- Script maintenance during agile cycles
- Test environment instability

17. What is your approach when a script fails in CI pipeline?

Check logs → validate environment → re-execute selectively → analyze root cause → fix script or raise defect.

18. How do you handle popups in Selenium?

Using:

- Alert interface
- Window handles
- `ExpectedConditions.alertIsPresent`

19. What is API Automation? Which tools do you use?

Automating APIs using:

- Postman (Collections + Newman)
- RestAssured for Java projects

20. How do you validate an API response?

Check:

- HTTP status
- Header
- JSON values

- Response time
- Schema validation

21. What is JSONPath?

Used to extract specific values from JSON responses.

22. What is CI/CD? How did you integrate automation?

CI/CD automates build → test → deploy.

Automation integrated using Jenkins + Git → triggers on commit → runs scripts → publishes report.

23. What is your Jenkins pipeline structure?

Checkout → Install dependencies → Run test suite → Generate report → Mail/Slack notification.

24. What is a Hybrid Framework?

Combination of POM + Data-driven + Keyword-driven + Utilities + Reporting.

25. How do you manage test data in automation?

Using:

- Excel
- JSON
- Properties file
- Database

26. Explain DataProvider in TestNG.

Used for parameterized testing — provides multiple sets of input data to a single test.

27. What is Cucumber? Why use BDD?

BDD tool using Gherkin syntax.

Used for:

- Better collaboration
- Human-readable scenarios
- Faster understanding for BA/PO

28. How do you maintain your automation scripts?

Version control (Git) → branching strategy → pull request review → refactoring → removing dead code.

29. If your build fails 10 minutes before a release, what will you do?

Stop full-suite run → execute only critical smoke tests → analyze failing script → verify manually → update stakeholders → fix in next sprint if non-critical.

30. What's the difference between driver.close() and driver.quit() commands?

The **driver.close()** command is used to close the current tab/window of the browser, which is controlled by the Selenium WebDriver.

The **driver.quit()** command closes all browser windows and ends the Selenium WebDriver session.

31. Explain the differences between driver.findElement() and driver.findElements() in Selenium WebDriver

driver.findElement() returns the first web element matching the provided locator strategy, while **driver.findElements()** returns a list of all elements that match the locator. The former is useful when you expect a single element, like clicking a button, and the latter is used when dealing with multiple elements, like a list of search results.

32. What is parameterization in Selenium?

It's the process of parameterizing test scripts (or executing the same test multiple times with different parameters) to automate the passing of data to a specific application during runtime. Qualified candidates should explain what parameterization is and how it functions in Selenium specifically. They should also mention that this process can be conducted several times with various values.

33. How do you get all the options in a dropdown in Selenium?

You can retrieve all options in a dropdown in Selenium by using the **getOptions()** method of the Select class. Candidates should be able to explain how this method retrieves all options on a Select tag and provides a list of web elements without accepting any arguments.

34. Explain the difference between getText() and getAttribute() in Selenium

The **getAttribute()** method provides the value of a particular HTML attribute, and the **getText()** method provides the visible text of a web element.

35. What are assertions in Selenium? What are the types?

Assertions are validations (checkpoints) to help the tester keep track of commands for an application. They help determine whether a test case is behaving as expected by validating the automated test cases so testers can understand if the tests have passed or failed.

Types:

1. **Soft asserts** help verify certain test conditions in the case even after the assert condition fails.
2. **Hard asserts** abort test execution when the test doesn't meet the asserted condition.

Hard Assertions: `assertNull()`, `assertNotNull()`, `assertEquals()`, `assertNotEquals()`, `assertTrue()`, `assertFalse()`

36. How would you handle dynamic elements that change attributes or IDs on each page load?

For dynamic elements, you must use robust locators like XPath or CSS selectors based on stable attributes or parent elements. Additionally, implement explicit waits to ensure the element is present before interacting, using ExpectedConditions to wait for specific conditions.

37. Explain how to handle browser cookies in Selenium WebDriver and some different scenarios it applies to.

Browser cookies are small data blocks stored by websites for session management, tracking, and personalization.

Key ways to manage:

1. **Retrieving:** `get_cookies()`
2. **Adding:** `add_cookie()`
3. **Deleting:** `delete_cookie()`
4. **Handling individual cookies:** grab specific cookie and change attributes
5. **Handling expiry time:** set cookie's expiry time

Scenarios: Logging in, Maintaining session status, Cookie-based function tests

38. Explain the concept of the FluentWait class in Selenium and how you would implement it.

FluentWait is an advanced version of WebDriverWait, allowing custom polling intervals and ignoring specific exceptions. It's useful for handling scenarios where elements might appear/disappear over time. To implement: create a FluentWait instance, set up its conditions and polling intervals, then use it to wait for an expected condition.

39. What is the difference between getWindowHandle() and getWindowHandles()?

getWindowHandle() returns the unique handle of the currently focused window or tab.

getWindowHandles() returns a set of all open window handles. Handy for multiple windows/tabs switching.

40. How do you perform drag-and-drop operations in Selenium WebDriver?

Use the Actions class: call clickAndHold() on source element, then moveToElement() to target, and finally release(). This simulates drag-and-drop effectively.

Python for Test Automation - Interview Q&A;

41. How do you use Python with Selenium WebDriver?

Install selenium package: `pip install selenium`

Use WebDriver: `from selenium import webdriver`

`driver = webdriver.Chrome()`

Python offers easy syntax, pytest/unittest frameworks, and rich libraries for automation.

42. What is pytest and why is it popular for automation?

pytest is a testing framework that requires less boilerplate than unittest.

Features: fixtures, parametrization, plugins, detailed assertion introspection, parallel execution with pytest-xdist.

43. How do you handle exceptions in Python automation scripts?

Use try-except blocks.

try:

`element.click()`

except NoSuchElementException:

`print('Element not found')`

Use finally for cleanup.

44. Explain Python decorators. How are they useful in testing?

Decorators modify function behavior without changing code.

Use case: `@pytest.fixture` for setup/teardown, `@allure.step` for reporting, retry decorators for flaky tests.

45. How do you read/write Excel files in Python for data-driven testing?

Use libraries:

- **openpyxl**: `wb = openpyxl.load_workbook('data.xlsx')`

- **pandas**: `df = pd.read_excel('data.xlsx')`

Iterate rows and feed to `@pytest.mark.parametrize`.

46. What is the difference between list, tuple, and set in Python?

- **List**: mutable, ordered, allows duplicates `[1,2,3]`

- **Tuple**: immutable, ordered `(1,2,3)`

- **Set**: mutable, unordered, unique elements `{1,2,3}`

47. How do you make API calls in Python for API testing?

Use **requests** library:

`import requests`

`response = requests.get(url, headers=headers)`

`assert response.status_code == 200`

Validate with `response.json()` and `jsonschema`.

48. What are Python virtual environments? Why use them?

venv isolates project dependencies.

`python -m venv venv → source venv/bin/activate`

Prevents version conflicts between projects.

49. How do you run Python tests in parallel?

With pytest: pip install pytest-xdist
pytest -n 4 → runs tests on 4 CPUs
Ensures thread-safe code and independent test data.

50. Explain the use of logging in Python automation frameworks.

Use **logging** module instead of print.

logging.info('Test started')

Configure levels: DEBUG, INFO, WARNING, ERROR

Output to file: logging.basicConfig(filename='test.log')

Playwright Interview Q&A;

51. What is Playwright and how is it different from Selenium?

Playwright is a modern automation library by Microsoft supporting Chromium, Firefox, WebKit.

Advantages: Auto-wait, network interception, faster execution, better handling of modern web apps, no WebDriver needed.

52. How do you install and launch a browser in Playwright Python?

pip install playwright

playwright install

Code: from playwright.sync_api import sync_playwright

with sync_playwright() as p: browser = p.chromium.launch(headless=False)

53. Explain Playwright's auto-waiting mechanism.

Playwright automatically waits for elements to be actionable before performing actions.

Checks: attached, visible, stable, enabled, editable.

Eliminates most explicit waits needed in Selenium.

54. How do you handle multiple tabs/windows in Playwright?

Use **context.new_page()** for new tab.

pages = context.pages → list all pages

page = context.wait_for_event('page') → capture popup

Switch with page.bring_to_front()

55. How do you perform API testing with Playwright?

Use **APIRequestContext**:

api_context = playwright.request.new_context()

response = api_context.get(url)

assert response.ok

Can intercept/mock: page.route('**/api', lambda route: route.fulfill(json={}))

56. What are Playwright locators? Why use them over CSS/XPath directly?

Locators are auto-waiting and retry-able.

page.locator('text=Login')

page.get_by_role('button', name='Submit')

page.get_by_test_id('username')

More resilient than raw selectors.

57. How do you capture screenshots and videos in Playwright?

Screenshot: page.screenshot(path='screen.png', full_page=True)

Video: browser = p.chromium.launch(record_video_dir='videos/')

Trace: context.tracing.start() → context.tracing.stop(path='trace.zip')

58. How do you run Playwright tests in parallel?

With pytest-playwright: pytest --numprocesses 4

Each test runs in isolated BrowserContext.

Configure in pytest.ini or conftest.py

59. Explain network interception in Playwright.

Mock APIs or block requests:

```
page.route('**/api/users', lambda route: route.fulfill(status=200, body='[]'))
```

```
page.route('**/*.png', lambda route: route.abort())
```

Useful for testing edge cases without backend.

60. What is Codegen in Playwright?

Playwright Codegen records user actions and generates test code.

Run: playwright codegen <https://example.com>

Outputs Python/Java/JS code with selectors.

Great for quick script bootstrapping.

[follow arun raj kathi for more interview guidance](#)